UNIVERSIDADE FEDERAL DE MINAS GERAIS
CENTRO DE CIÊNCIAS EXATAS
DEPARTAMENTO DE COMPUTAÇÃO
DISCIPLINA: Algoritmos II
DOCENTE: Renato Vimieiro

Trabalho Prático 1

Jean Berly Rivera Sarmiento
Raissa Lauar Navarro

Belo Horizonte-MG, 16 de outubro de 2026

1 Introdução

O problema apresentado consiste em desenvolver uma aplicação web para a visualização e consulta de entregadores em operação na cidade de Belo Horizonte. O mapa interativo permite a visualização do restaurante selecionado juntamente com a disposição dos entregadores, suas distâncias e estimativa do tempo de chegada. A implementção foi realizada em python com as bibliotecas dash-leaflet, osmnx e pandas, que permitiram a construção da interface, o uso do grafo da malha viária do OpenStreetMap e a manipulação do csv com os bares e restaurantes.

bibliotecas utilizadas: dahs, dash-leaflet, osmnx, pandas, scipy, shapely, scikit-learn.

2 Construção da interface gráfica

A aplicação web foi construída utilizando a biblioteca dash/dash leaflet, que integra elementos de html e css. Para a obtenção dos inputs do modelo, como o restaurante/bar escolhido, o tamanho do raio de busca e a quantidade de entregadores, foram usados select e inputs.

A interface também apresenta dois botões, um de *Gerar Entregadores*, de acordo com a Normal Mutivariada, que plota no mapa os seus pontos de ocorrência em cinza. Em seguida é possivel clicar no botão de *Filtrar Entregadores* que insere os entregadores na Tree, calcula suas distâncias, tempo de chegada e filtra pelos que estão dentro do raio pedido, representados em vermelho.

No tratamento de dados, nem todos os estabelecimentos possuem nome fantasia. Assim, no select é exibido primeiro os estabelecimentos pelo seu nome fantasia, seguido do seu bairro (para diferenciação entre unidades) e o seu endereço. Ao acabar essa enumeração, mostra os estabelecimentos com seus nomes jurídicos e endereços (são os bares e restaurantes que não possuem nome fantasia cadastrado). Se ele apresentar nome fantasia composto de somente caracteres especiais, o nome apresentado é o jurídico. Segue um exemplo:

id	CNAE PRINCIPAL	DESCRIC	DESC L	NOME LOGRADOURO	NUMERO IMO	COMPLEMENTO	NOME BAIRRO	NOME	NOME FANTASIA

Figura 1: Header do csv

319291	5611201	RESTAUR	RUA	RIO DE JANEIRO	1651	LOJA:A,	LOURDES	JULIANE BENTO ALVES	*****

Figura 2: Uma linha do csv

2.1 Coordenadas dos bares e restaurantes

A latitude e longitude dos bares e restaurantes foi obtida através do site da Prefeitura de Belo Horizonte. O dicionário de dados da atividade econômica fornece a geometria do local que pode ser convertido em latitude e longitude. Para essa conversão, utilizamos o software qgis, trabalhado nos laboratórios de geoprocessamento.

3 Grafo da malha viária de Belo Horizonte

Para dar suporte as buscas no mapa de Belo Horizonte, foi permitido usar a biblioteca osmnx para a estrutura do mapa de Belo Horizonte.

3.1 Obtenção da malha viária

Para obter o mapa de Belo Horizonte, foi utilizado o site OSM By the Slice que permite baixar o mapa fornecido pelo OpenStreetMap. Com isto, utilizamos a biblioteca osmnx para obter o grafo de Belo Horizonte a partir desse arquivo e salvar na memória em disco.

3.2 Correção do Grafo

O mapa obtido da malha viária de BH contém o comprimento e geometria das ruas(arestas). Porém, não contém em todas as arestas a informação do limite máximo de velocidade da via. Já que essa informação foi necessária na parte de calcular o tempo de chegada do entregador ao restaurante, criamos o script trataGrafoVelocidadeArestas.py, que basicamente preenche essa informação para as arestas que não tem essa informação com base na média das outras arestas do grafo. Essa média ficou em cerca de 42km/h.

3.3 Compactação do grafo

O arquivo do grafo de BH ultrapassa os 100mb permitidos pelo github. Para resolver esse problema, no GitHub se encontra uma versão compactada do arquivo que é descompactada antes da inicialização do servidor web.

4 Implementação da tree compacta

4.1 Nó

Para simular o nó da árvore foi criada uma classe *No* que possui o prefixo do hash, os nós filhos e os ids dos entregadores que possuem o hash. Além disso, apresenta uma função que verifica se o nó é final, ou seja, se não possui filhos.

4.2 Árvore

Para a árvore, criamos a classe *Tree*, que na sua inicialização gera o nó raiz vazio.

4.2.1 Inserir nó

A função de inserção recebe o id do entregador e suas coordenadas, gerando o geohash correspondente. Caso não exista um filho associado ao primeiro caractere do hash, cria um novo nó contendo o hash completo e o ID do entregador. Caso já haja, compara os prefixos para identificar a parte em comum entre os hashes.
Se todo o prefixo do nó filho corresponder ao início do hash inserido, o ID do entregador é adicionado à lista de IDs desse nó. Se ainda houver caracteres a serem processados, a inserção continua recursivamente nos filhos. Como os ids são armazenados nos níveis superiores, o custo da busca é diminuído, o que será abordado em seguida.
Se os prefixos forem diferentes, é criado um nó intermediário com a parte em comum. O nó antigo é atualizado, e um novo nó é criado para armazenar o restante do hash inserido.

4.2.2 Buscar entregadores

Para a busca dos entregadores, a função *buscarIds* percorre a árvore comparando com os prefixos dos nós. Se encontrar um nó com todo o hash, ele retorna a lista de entregadores desse prefixo, que corresponde a todos os seus filhos. Como os ids dos entregadores são armazenados nos nós intermediários, não é preciso percorrer toda a subárvore, o que faz com a complexidade da busca seja $O(m)$, com m sendo a atura da arvore, pois tem somente o custo de achar o hash na árvore e nenhum custo de achar os ids. Logo, temos $O(m) < O(m + k)$.

5 Simulador de Geração de Entregadores

5.1 Especificação demandada

Foi especificado que dado um restaurante deveria-se gerar N entregadores ao redor dele. Para isso deveria se gerar as suas posições seguindo uma normal multivariada. Além disso, também deveria ser garantido que a posição deles é válida, ou seja, que os entregadores estivessem sobre ruas e não dentro de prédios ou lagoas, dessa forma a sua posição gerada a partir da normal deveria ser ajustada posteriormente para a malha viária.

5.2 Normal Multivariada

Decidimos usar uma normal 2 bivariada, onde as dimensões seriam a latitude e a longitude.

5.2.1 Parâmetros da Normal Bivariada

Foi escolhido a média como sendo a latitude e a longitude do restaurante e para a matriz de variância escolhemos $0.0005I$, com I sendo a matriz Identidade 2x2. O fator 0.0005 foi escolhido empiricamente, para garantir que os entregadores não ficassem muito distantes e nem muito pertos.

5.3 Ajuste das Coordenadas

Para ajustar as coordenadas dos entregadores para posições válidas foi utilizado métodos da biblioteca osmnx. Entraremos em detalhes nessa lógica agora.

5.3.1 Função simulaGeracaoEntregadores

Basicamente, a função simulaGeracaoEntregadores constroi a normal bivariada utilizando o scipy e amostra N entregadores. A partir destes, retornamos a função ajustaPosicaoEntregadores

5.3.2 Função ajustaPosicaoEntregadores

Para ajustar a posição dos entregadores seguimos alguns passos para cada entregador:

1. Encontrar a rua mais próxima do entregador
2. Encontrar qual ponto nessa rua está mais próximo do entregador
3. Atualizar a posição do entregador

Veja que a biblioteca osmnx nos fornece a função nearest_edges que é responsável pelo item (1). Para o passo (2), como o osmnx usa a biblioteca Shapely para guardar a geometria das ruas, basicamente projetamos com a função shapely.LineString.project a posição do entregador na rua mais próxima dele. Com isso encontramos a sua posição ajustada e podemos realizar (3).

6 Seleção dos entregadores dentro do raio

6.1 Função gerar geoHash

A função de gerar o geohash considera o intervalo de longitude entre -180 e 180 graus, e o de latitude de -90 e 90 graus, fazendo divisões sucessivas desse quadrado. Em cada divisão, é analisado se a coordenada está na direita ou esquerda, adicionando um bit (0 para esquerda/baixo) e 1 (direita/cima). Em seguida, os bits são agrupados de cinco em cinco e são convertidos em caracteres do alfabeto Base32. Como temos um geohash de tamanho 8, são gerados 40 bits, correspondentes a 40 divisões.

6.2 Cálculo da distância em linha reta

Para o cálculo da distancia entre pontos, ou seja, do entregador e do restaurante, utilizamos a fórmula de Haversine. Ela leva em consideração a inclinação da Terra e as coordenadas geográficas. Primeiro, as distâncias entre as coordenadas foram convertidas para radianos. Em seguida, o valor de a foi calculado e é usado para converter em kilometros considerando o raio da Terra.

$$a = \sin^2\left(\frac{\Delta lat}{2}\right) + \cos(lat_1)\cos(lat_2)\sin^2\left(\frac{\Delta lon}{2}\right)$$

$$d = 2R \cdot \arcsin\left(\sqrt{a}\right)$$

7 Estimativa do Tempo de Chegada ao restaurante

Para estimar o tempo de chegada entre os entregadores e o restaurante foi utilizado Dijkstra em cima do grafo gerado pelo osmnx. É importante ressaltar que o osmnx utiliza de um multigrafo orientado, logo entre dois vértices pode existir mais de uma aresta.

7.1 Min-Heap

Para servir de apoio ao algoritmo de Dijkstra foi utilizado um min-heap. Basicamente, um min-heap é uma árvore binária que mantém a propriedade de que os nós pais são sempre menores que seus filhos. Além disso, guardamos a árvore binária num array. A vantagem é que conseguimos inserir e remover nós(só conseguimos removemos o menor) no min-heap em tempo logaritmo no número de nós na árvore.

7.1.1 Remoção de valores

Para remover um elemento (a raiz da árvore binária) o que fazemos é colocamos o último elemento do array na posição da raiz e corrigimos sua posição na árvore. Dessa forma mantemos a invariante do Min-Heap que pais são sempre maiores que seus filhos

7.1.2 Inserção de valores

Para inserir um valor colocamos ele na última posição do array e vamos subindo ele na árvore. Dessa forma mantemos a invariante do Min-Heap que pais são sempre maiores que seus filhos

7.2 Dijkstra

A ideia do algoritmo de Dijkstra para encontrar o caminho entre um vértice origem e um destino é de inicializar a distância entre a origem e todos os outros vértices do grafo como infinito e para ele mesmo como zero. A partir disso, guardamos as duplas (distancia, vértice) no Min-Heap e vemos qual é o mais perto (menor elemento no Heap) e vemos se a distância para os seus vizinhos é menor que a menor distância calculada até o momento, se sim, atualizamos essa distância e inserimos o par (novaDistancia, vizinhoDoVerticeMaisPerto) no Min-Heap.

7.2.1 Complexidade

Como só visitamos cada aresta uma única vez e iteramos no pior caso em todas as E arestas e o custo ao visitar cada aresta é a inserção/remoção do heap temos a complexidade de $O((|E| \log V)$, onde E é o número de arestas e V é o número de vértices. (Assumindo o grafo conexo, já que estamos tratando de uma malha viária de uma cidade.)

8 Experimentos práticos

Os tempos calculados para cada consulta foram baseado apenas na execução do código relativo a lógica e não foram levados em conta o tempo para atualizar a interface (parte do dash-leaflet.)
Para calcular o tempo foi utilizado a biblioteca time.

8.1 Exemplo da interface do sistema:

Selecionamos o restaurante 021 Burguer, geramos 20 entregadores e calculamos os que estão em um raio de 2km do local.

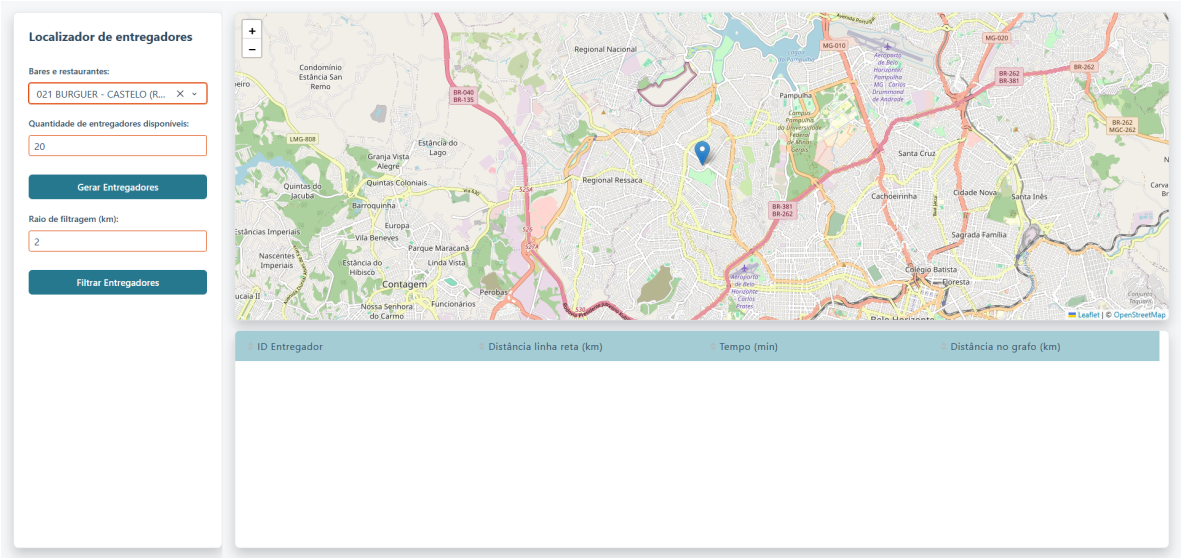


Figura 3: Interface ao selecionar um restaurante

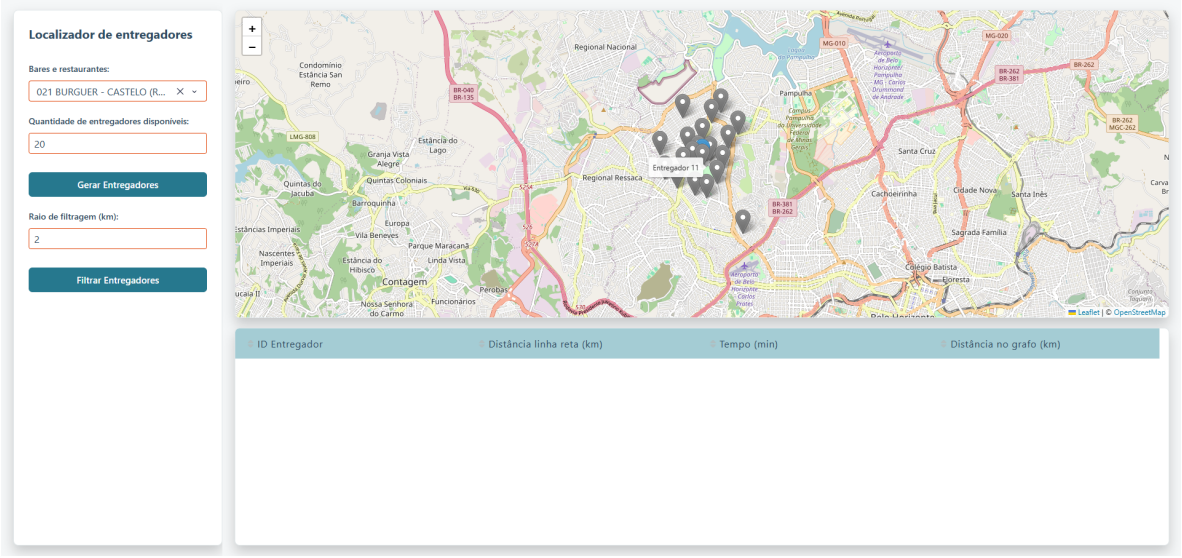


Figura 4: Interface ao gerar 20 entregadores (demorou 2.43s)

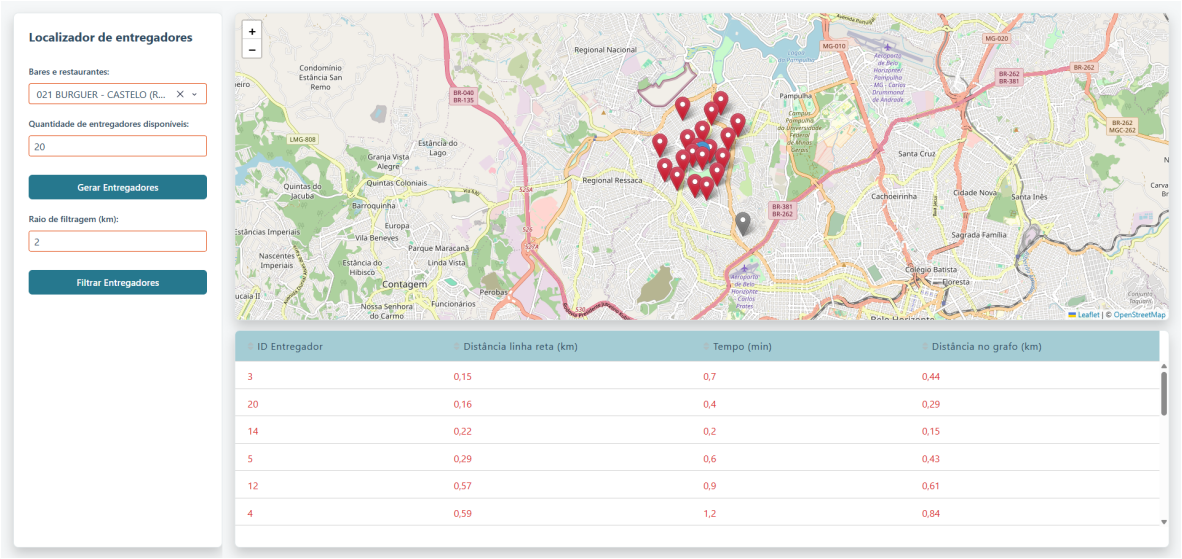


Figura 5: Interface ao filtrar e calcular tempo ate o restaurante de 20 entregadores no raio de 2km (demorou 0.34s)

8.2 Casos de exemplo

Tabela 1: Tempo de processamento de acordo com o número de entregadores com filtro de 2km.

Número Entregadores	Geração e Ajuste	Ajuste	Filtro e tempo até o restaurante
10	2.402334690093994s	2.400908946990967s	0.3206007480621338s
50	2.697873830795288s	2.6954824924468994s	0.5395069122314453s
100	2.9462225437164307s	2.9424023628234863s	0.6578900814056396s
200	2.7468128204345703s	2.7393429279327393s	1.440568208694458s
1000	2.7385528087615967s	2.6963231563568115s	4.349797964096069s

Vemos que quando aumentamos o número de entregadores rapidamente o custo de filtrar e calcular o tempo dos entregadores até o restaurante é o que mais aumenta. Isso se dá porque temos que calcular o djikstra para cada entregador até o restaurante. O custo fixo do ajuste da posição dos entregadores é relevante também mas não cresce tanto com o número de entregadores.

9 Referências

Rosen, Kenneth H. Matemática Discreta e Suas Aplicações. 6ªEdição. Editora Mc Graw Hill, 01/01/2009.
https://medium.com/@artemkhrenov/database-indexing-strategies-b-tree-hash-and-specialized-indexes-explained-95a3e5e3b632
https://www.geeksforgeeks.org/system-design/geohashing-and-quadtrees-for-location-based-services/
https://www.movable-type.co.uk/scripts/geohash.html
https://www.dash-leaflet.com/
https://www.igismap.com/haversine-formula-calculate-geographic-distance-earth/
Introduction to Algorithms: Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein.
Slides do docente